

T24 Componentisation

R17



T24 Componentisation



Amendment History:

Revision	Date Amended	Name	Description
1	30 th October 2015	T. Aubert	Initial version
2	15 th November 2015	T. Aubert	Updates
3	4 th December 2015	H. Aubert	Format & Review
4	17th Feb 2015	T. Aubert	Add Paragraph for Interfaces
5	15th Mar 2016	M.kumar	R16 Review



Copyright

Copyright (c) 2014 TEMENOS HOLDINGS NV
All rights reserved.

This document contains proprietary information that is protected by copyright. No part of this document may be reproduced, transmitted, or made available directly or indirectly to a third party without the express written agreement of TEMENOS UK Limited. Receipt of this material directly TEMENOS UK Limited constitutes its express permission to copy. Permission to use or copy this document expressly excludes modifying it for any purpose, or using it to create a derivative therefrom.

Errata and Comments

If you have any comments regarding this manual or wish to report any errors in the documentation, please document them and send them to the address below:

Technology Department

Temenos Headquarters SA
2 Rue de l'Ecole-de-Chimie,
CH - 1205 Geneva,
Switzerland

Tel SB: +41 (0) 22 708 1150

Fax: +41 (0) 22 708 1160

Please include your name, company, address, and telephone and fax numbers, and email address if applicable. TAFJdev@temenos.com



Table of Contents

Copyright.....	2
Introduction.....	4
The benefits.....	4
Creating a new Component.....	6
The notion of « module ».....	7
Creating a new Component (continuation).....	7
The Scoping.....	9
public.....	9
module.....	9
private.....	9
external.....	9
The Method.....	9
The parameters.....	10
The jBC information.....	11
The Constant.....	12
The Table.....	12
The property.....	13
Compiling a component.....	15
Doing tests before implementing.....	15
Using a Component.....	18
The Methods.....	18
The Constants.....	20
The Properties.....	21
The Tables.....	24
Documentation.....	26
Implementing methods.....	27



Introduction

A component is a logical and technical grouping of jBC artefacts. By « jBC artefacts » we mean subroutines, functions, commons and tables. This grouping was already existing, but by naming conventions only. As an example, the routine EB.ACCOUNTING was part of « EB » by it's name. But what about EXCHRATE ?

Since version R15, a new type of file has been released : the .component.

This file will contain a list of all the artefacts mentioned earlier. This file will be compiled as a normal jBC routine and will be used during the development.

This .component is the façade to the underlying implementations. This acts as a king of « interface ».

The benefits

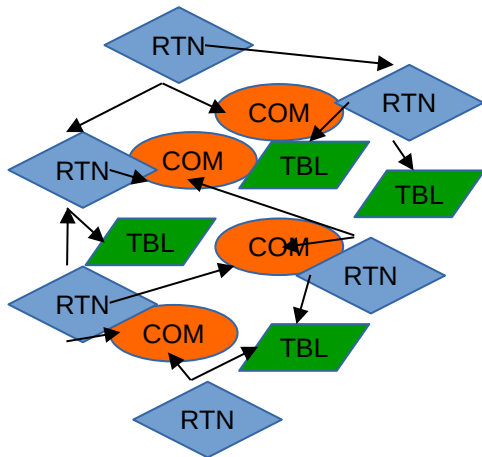
Before going in details into the component world, let's talk about the benefits of it.

- Grouping code means that the inter-dependencies between the different groups becomes visible and manageable.
- COMMON variable access is done through getters and setters. No more side-effects of changing a common variable without knowing it.
- Tables access are restricted. Cannot write in a table in a different component
- Constants (EQU) are defined in the component and not copied across like a \$INSERT does. So There are no risk to clash with different versions.
- A component is self-describing. Protect user to have to know the underneath implementations
- Auto-completion in jBC Editor
- Scoping allow method, properties, ... to be hidden from the external world (an other component)
- Defining Component oriented Unit Tests (UnitTest Framework)

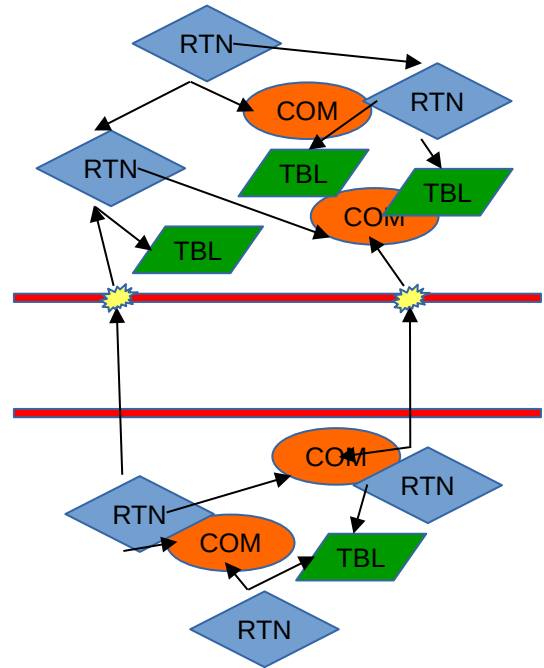
T24 Componentisation



This picture presents the same kind of code, without and with a « componentisation ». It clearly shows that, once the components are well defined, the inter-actions between them becomes obvious and manageable.



No components



With components

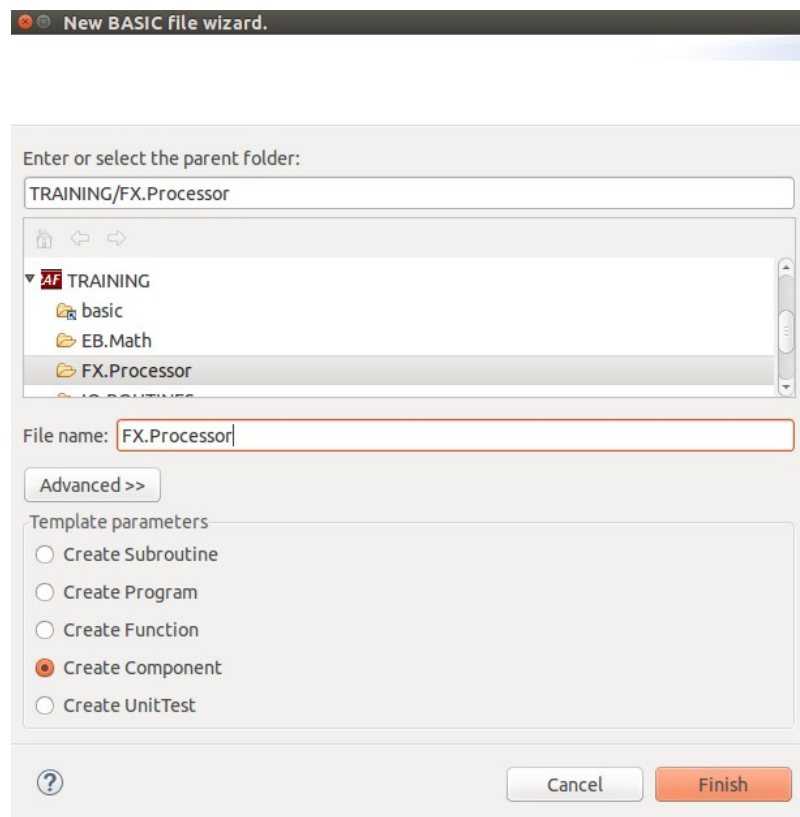
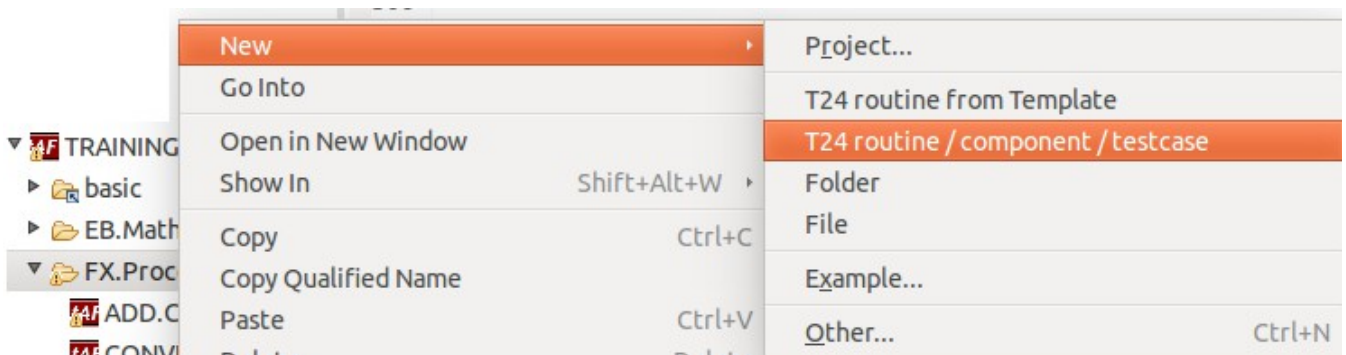


Creating a new Component

In EDS, creating a new component is done the same way as creating a new SUBROUTINE.

In the TAFJ Perspective

Right-click on the directory you want the component to be, choose New → T24 routine / component / testcase, enter a name for your component and click finish.



Important: A component

name is composed of a «module».«component»



The notion of « module »

A module name is usually a 2 upper-case letter (eg « AA », « FT », « EB ») in the component name.

A Module is a logical group of component. This is important to consider the module as it has a real implication in the scoping (see « Scoping » chapter).

Creating a new Component (continuation)

Once you have made a new → T24 component, you will automatically enter the EDS Component editor.

```
component FX.Processor
```

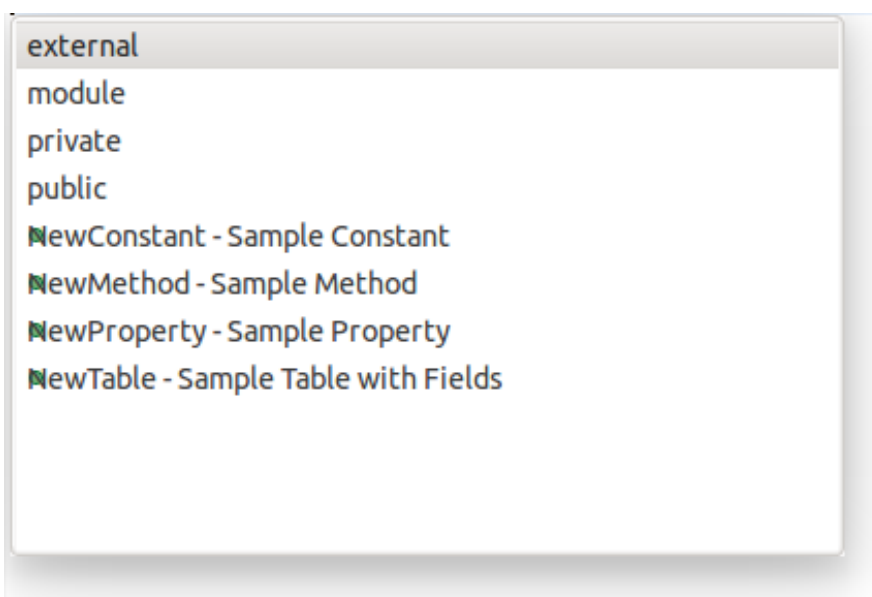
```
# Component Isolation Definition
```

```
metamodelVersion 1.6
```

```
# -----
```

The DesignStudio/Eclipse Component editor comes with auto-completion facilities. "

TO do so, press Ctrl + <SPACE>. You will have something like this:





The last for options are the template. So choosing, as an example, “NewMethod” will automatically add this code ...

```
module method OperationName
(
    IN ArgumentName string [0..1]
)
{
    jBC: RoutineName
}
```

With the focus to the scope. You can navigate from one field to another one with the <tab> / <Shift> + <Tab>.

Before going in details on the 4 available options, lets discuss the Scoping.



The Scoping

The Scope is a mandatory information for ALL the component artefacts (Constant, Method, Property and Table). It can have 4 different values :

public

The artefact is visible from everywhere (other module, other component)

module

The artefact is visible only for the component being part of the same module

private

Only visible in the same <module>.<component>

external

This option is for future release and is useless for now.

This is important to consider the scoping with care, and not make everything public. By default the scoping is « module », but reducing it to « private » and change it only if necessary is certainly a good practice.

The Method

A method is describing a jBC SUBROUTINE or FUNCTION. It is composed of the scope, an alias Name (OperationName), the optional return type (if a function) the list of the parameters and the jBC implementation name (the « real » SUBROUTINE / FUNCTION name)

module method OperationName	module method OperationName :
(	string (
<<params>>	<<params>>

Currently (Nov 2015) only “string” is a valid return type.



The parameters

A parameter is composed of an Access, it's name it's type and it's multiplicity.

There are coming right after the name of the method, are separated by coma and wrapped in brackets (...)

The access is IN, INOUT or OUT.

IN means the parameter is not modified in the routine

OUT means it is only for returning a value

INOUT means the parameter value is relevant and will be modified.

As today, this is an information only. In future releases, this value will be verified at compile time.

The name is any valid variable name. This is encouraged to use the camel casing.

The type define the type of parameter. As per today, only « string » is valid.

The type can be followed by open/close bracket to define a DIMentionned array.

Example:

```
(  
    IN FirstParam string,  
    INOUT SecondParam string,  
    OUT AnArray string()  
)
```



The jBC information

The jBC information define the « real » SUBROUTINE / FUNCTION name.

It is wrapped in curly brackets { ... }

Example :

```
{  
    jBC: RoutineOrFunctionName  
}
```

A Full Method :

```
module method OperationName  
(  
    IN FirstParam string,  
    INOUT SecondParam string,  
    OUT AnArray string()  
)  
{  
    iBC: MY.ROIITTNF.NAMF
```

or

```
module method OperationName : string  
(  
    IN FirstParam string,  
    INOUT SecondParam string,  
    OUT AnArray string()  
)  
{
```



The Constant

The constant is quite easy to implement; it is composed of the scope, a name and a value. Optionally, we can specify a JBC name which is only necessary for auto-incorporation (see specific chapter)

Example:

```
module constant ConstantName = 10

module constant ConstantName(JBC.NAME) = 10
```

The Table

The table is composed of the scope, an alias (the tableName), the real JBC table name (without the prefix) and a list of fields.

The fields are separated by comma and wrapped in a fields:{ } group. Note that the fields can have, for the same reasons as constants, the JBC name in brackets.

Note that there is no comma separator between the fields.

Example:

```
module table TableName
{
    t24: MY.TABLE
    fields: {
        Field1 = 1
        Field2(JBC.NAME) = 2
    }
}
```



The property

As mentioned earlier, a property is wrapping a COMMON variable.

A Property is composed of the scope, the property name, it's access, it's type and optionally the corresponding COMMON defined by an Insert file and a variable name.

The access can be read, write, readwrite or readonly(deprecated).

If read, only a getter will be available.

If write, only a setter will be available.

You can, by separating read / write for the same property fully manage the scoping. For example, you can decide to have your property private for the write, and public for the read.

public property read PropertyName

private property write PropertyName

If the COMMON is a DIMmed array, the jBC variable name must be followed by the number of dimensions in-between bracket. Notice that we are talking about the NUMBER of dimensions, not the size!

R.NEW(500) has 1 dimension

MY.ARRAY(10,20) has 2 dimensions

Example:

```
module property readwrite PropertyName : string {  
    jBC: I_FX.Processor ->CommonName  
}
```

```
module property readwrite RNew : string {  
    jBC: I_COMMON ->R.NEW(1)  
}
```



A property can also not have any JBC: implementation. In this case, the underlying COMMON is only accessible from the property. This will be the favourite option for new components.

Example:

```
module property readwrite PropertyName
```

There is no such support for DIMmed array for the properties not having a "jBC:" information.



Compiling a component

A component, like any other jBC file must be compiled. In EDS, simply saving it will invoke the jBC compiler so you should not have to worry about it. You always can right-click on the file and do a « compile Basic file »

Doing tests before implementing

Once your component has been saved and compile, you can directly start using it. Of course, if the jBC implementation has not been done, this will fail at runtime, but not at compile time, and most important, not for Unittests.

This mean that you can define a component, create tests for it, and implement after having created the test!

This is a very important point, and we will quickly make a step-by-step procedure to illustrate it.

- 1) Create a new component, call it MY.Component
- 2) Ctrl + <Space> add a new method, keep the default values, save

Here you have created a method MY.Component.OperationName(...) with one parameter. This method is supposed to invoke the jBC routine « RoutineName »

```
component MY.Component
metamodelVersion 1.6
module method OperationName
(
    IN ArgumentName string [0..1]
)
{
    jBC: RoutineName
}
```



3) Create a UnitTest, call it **TestOperationName**, and edit it like this :

```
TESTCASE TestOperationName

    UTF.setTarget("MY.Component.OperationName")

    P1 = 123

    UTF.addParam(P1)

    UTF.runTest()

    UTF.assertEquals(P1, 246)

END
```

4) Save right-click on TestOperationName.tut (the test file) and run as ... Basic UnitTest

Failure			
Completed.			
Test	Status	Description	Result
TestOperationName.tut	Completed	'123' <> '246'	Failure

Obviously, the test is failing. Now change hat, take the developer one, and implement the method « RoutineName »



5) Create a new SUBROUTINE, call it **RoutineName**, and edit it like this :

```
SUBROUTINE RoutineName(P1)
```

```
    P1 = 2 * P1
```

```
END
```

. . . and re-run your test. You don't have to recompile anything, just re-run it.

Success			
Completed.			
Test	Status	Description	Result
TestMyRoutineName.tut	Completed	NA	Success



Using a Component

The Methods

In order to use a component, you have to define it with the `$USING` keyword

As an example to use the component we created in the previous chapter, create a new PROGRAM, call it MAIN, and start editing it like this :

```
PROGRAM MAIN

$USING MY.Component

END
```

First you will notice the auto-completion right after you typed `$USING ....`

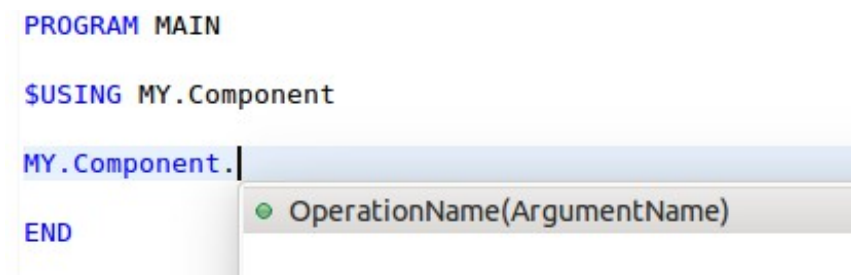
Then, you can type `MY. ....` and the auto.completion is proposing you « Component », and then `. ....` and nothing any-more !!!!

Why?

Look carefully at the component. It has only one method, and its scope is « module ». This means that only pieces of code in the same module can access the method.

So 3 solutions in our example:

1) Make your method public (edit the .component, save, come back on your code, type `MY.Component. ....` and you will see



T24 Componentisation



2) Put back « module » and use the 2nd option : make your program being part of the « MY » Module.

To do so, add the \$PACKAGE at the top of your program, and specify a name having « at least » the same module then your component. You can remove the \$USING clause.

```
$PACKAGE MY.Component  
  
PROGRAM MAIN  
  
MY.Component.|  
END
```

● OperationName(ArgumentName)

I mentioned a 3rd option well, not exactly. You can try to type OperationName yourself, finally, this is just auto-completion, right ? Wrong ! The compiler will refuse it, and this small exercise is describing how we are enforcing the scoping and why this is so important to define a proper scope. This feature is not only in Eclipse (EDS) but also for console compilation. No way to bypass it.

This means that for future releases, you only have to care about back-compatibility of public (evt module) methods.

```
1 PROGRAM MAIN  
2  
3 $USING MY.Component  
4  
5  
6  
7 END
```

✗ OperationName forbidden acces : module

But say you won't abandon so easily, and decide to call the underneath routine directly no no !

```
1 PROGRAM MAIN  
2  
3 $USING MY.Component  
4  
5  
6  
7  
8
```

✗ CALL RoutineName(2)
Forbidden direct CALL.
Must be : MY.Component.OperationName(ArgumentName) instead.



The only way is to remove the \$USING, but in that case you are back in the « old world » and there is no component involved any-more.

Now that you understood how the component is invoked, and how the Editor is helping you to auto-complete, let's explore the 3 remaining artefacts:

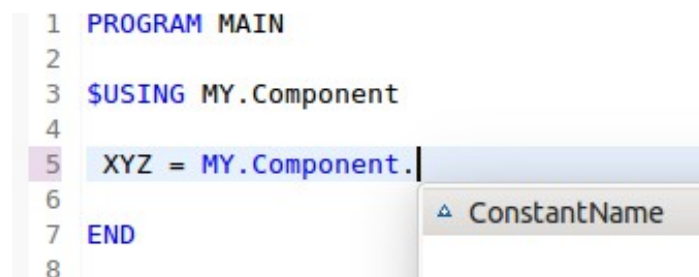
1. Constants
2. Properties
3. Tables.

The Constants

This is quite obvious, and in order to illustrate it, just add a constant declaration in MY.Component, make it public !, get back to your MAIN program, and type

MY.Component. ... And you will see your constant!

```
1 PROGRAM MAIN
2
3 $USING MY.Component
4
5 XYZ = MY.Component. |
6
7 END
8
```



Nothing else to add, apart maybe that if you change the value of your constant, you do not need to recompile your program to have the new value. This is a big difference comparing to the EQU in jBC.



The Properties

As mentioned previously, a property is mapping a common variable. The small difference is that you do not have a « pointer » to the variable, but the value itself.

Let's create a property.

First, create a I_ File (new File) (let's call it I_TEST.INSERT) and edit it like this:

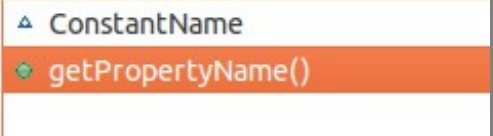
```
COM/MY.COMMON.BLOCK/MY.COMMON
```

Then, edit your component, and add a property :

```
public property read PropertyName : string
```

Back to you main program:

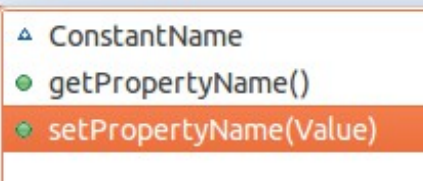
```
1 PROGRAM MAIN
2
3 $USING MY.Component
4
5 MY.Component.
6
7 END
8
```





There is no way to change the property, and the reason for it is because the property is readonly. Change it in the component to readwrite (double-click on readonly, Ctrl + <space> will show the list of possible values) or simply replace « readonly » by « readwrite », then back to your program :

```
1 PROGRAM MAIN
2
3 $USING MY.Component
4
5 MY.Component .
6
7 END
8
```



If a property is defined with array size, the getters and setters will act accordingly and new helper methods will be made available. To describe it, first, create a I_ File (new File) (let's call it I_TEST.INSERT) and edit it like this :

```
COM/MY.COMMON.BLOCK/MY.COMMON(50,50)
```

We have to do it because, as specified in the “The Property” chapter, a property without the “jBC:” block cannot be a DIMmed array. Then change your component to something like this:

```
public property readwrite PropertyName : string
{
    jBC: I_TEST.INSERT -> MY.COMMON(2)
}
```



If you get back to your MAIN program:

```
1 PROGRAM MAIN
2
3 $USING MY.Component
4
5 MY.Component.
6
7 END
8
```

- △ ConstantName
 - clearPropertyName()
 - getDynArrayFromPropertyName()
 - getPropertyName(idx1, idx2)
 - setDynArrayToPropertyName(dynArray)
 - setPropertyName(idx1, idx2, Value)

We can find our getter and setter, but also 3 new methods :

clear<property_name> is the JBC Equivalent to MAT MY.COMMON = "

getDynArrayFrom<property_name> is returning « ret » after something like this :
MATBUILD ret FROM MY.COMMON

setDynArrayTo<property_name> is the JBC Equivalent to MATPARSE MY.COMMON
FROM dynArray



!! WARNING !!

This is not possible to mix the same property with and without the "jBC:" setting.



This code is **INVALID**. The reason for it is that the getter (property read) points to something else than the setter (property write)

```
public property read  PropertyName : string
private property write PropertyName : string
{
    jBC: I_TEST.INSERT -> MY.COMMON
}
```

The Tables

Let's create a table definition in our component to illustrate it :

```
public table TblCurrencyRate
{
    t24: CURRENCY.RATE
    fields: {
        Currency = 1
        Rate = 2
    }
}
```

Back to MAIN program. You will notice your table name. Also, a method called « table » + your table name. Ignore this last one, it is here just for back-compatibility. Select your table name:

T24 Componentisation



```
1 PROGRAM MAIN
2
3 $USING MY.Component
4
5 MY.Component.TblCurrencyRate.
6
7 END
8
```

- CacheRead
- Currency
- Exists
- ExistsHis
- ExistsNau
- Rate
- Read

```
1 $PACKAGE MY.Component
2
3 PROGRAM MAIN
4
5 MY.Component.TblCurrencyRate.
6
7 END
8
```

- ExistsNau
- Rate
- Read
- ReadHis
- ReadNau
- ReadU
- ReadUHis
- ReadUNau
- Write
- WriteHis
- WriteNau

All the necessary (self describing) methods are here.... All ? Where are the write facilities ? In fact, there are « private » only. No one apart my component is allowed to write to my tables. If I want to provide some update facilities, I

will have to make public methods.

Nevertheless, let's cheat and make our MAIN program part of MY.Component to see these methods. Remember how to do it ? Add \$PACKAGE MY.Component and remove the \$USING.

Try again to Ctrl + <space> after TblCurrencyRate :

Delete, ReadU and Write are now available.



Note that the fields (defined as constant) are always public. This means that you can make a table private, create a method which return a part of the record only, and the user of your method will still be able to access the fields using the <MOD>.<Component>.<TableName>.<FieldName> methodology.



The interfaces

The components have a last type of definition, the interfaces.

You define an interface like this :

```
public interface NoParam (  
)  
  
public interface OneParam (  
    INOUT Overridelist string  
)
```

Interfaces are used to invoke methods « by name » (same as the old « CALL @ »)

Basically, an interface is nothing else than a method, but without the { ... jBC : ... } body.

Once an interface is defined, you can invoke it the same way as a method with one major difference : You specify the name of the method you want to invoke.

To describe it, let's imagine this full example :

```
/*  
 * Public interface with one param  
 */  
public interface CallByNameOneParam (  
    INOUT oneParam string  
)  
/*  
 * Real method pointing to jBC  
 */  
private method doSomething (  
    INOUT MyParam string  
)  
{  
    jBC: DO.SOMETHING  
}
```

You then can invoke « doSomething » by name like this :

```
TEST.Interface.CallByNameOneParam("doSomething", "param1")
```



Documentation

This is important to document your component. The user of it won't have it. All the auto-completion features are coming from the compiled component, not from the source.

Documenting is not only important for the public artefacts, but also for the private, the module, etc ... Here is a fully documented component:

```
component MY.Component
metamodelVersion 1.6
/*
 * This is a very important method
 * doubling the value passed
 */
module method OperationName
(
    /*
     * This argument is quite important
     * This should be a number.
     */
    IN ArgumentName string [0..1]
)
{
    jBC: RoutineName
}
/*
 * The value of this constant is 10
 */
public constant ConstantName = 10

/*
 * This property is just here for fun.
 * This is a double dimentioned array
 * absolutely useless.
 */
public property readwrite PropertyName : string
{
    jBC: I_TEST.INSERT -> MY.COMMON(2)
}

/*
 * This table is a 2 column table
```



```
*/  
public table TblCurrencyRate  
{  
    t24: CURRENCY.RATE  
    fields: {  
        Currency = 1  
        Rate = 2  
    }  
}
```

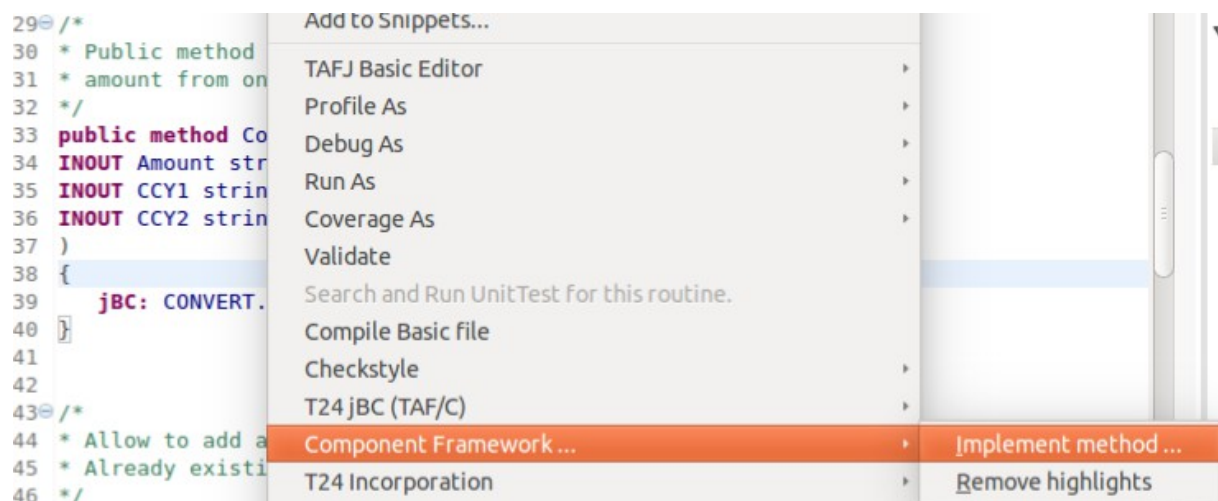
Implementing methods

The component editor has a special contextual menu to help developers to implement a method.

This menu is only available when the cursor is located in a method.

To illustrate it, position your cursor in a method, and right-click.

The following menu will offer you the possibility to implement the method:





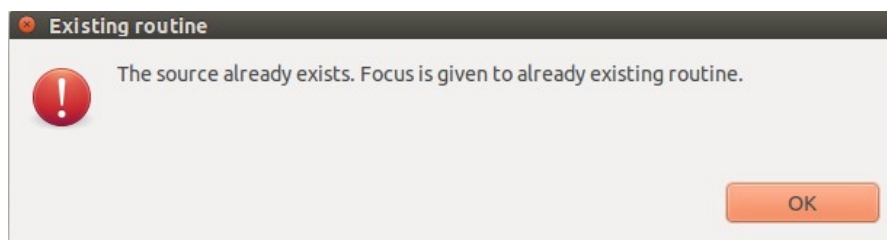
And clicking on “Implement method ...” have 3 possible action :

1. everything is fine, a new method will be implemented, and the focus given on it like this:

```
$PACKAGE FX.Processor
*
* Implementation of FX.Processor.Convert
*
* Amount(INOUT) :
* CCY1(INOUT) :
* CCY2(INOUT) :
*
FUNCTION CONVERT.AMOUNT(Amount, CCY1, CCY2)

RET = ''
RETURN RET
```

2. The second possibility is that the method already exists in the source. In that case, if the source is located somewhere else than the place where the component is, a warning will be given, the source file will be opened. If the source is located at the same place, the file is simply opened.



```
$PACKAGE FX.Processor

FUNCTION CONVERT.AMOUNT(AMT, CCY1, CCY2)

DEFFUN ROUND.AMOUNT

rate1 = FX.Processor.getRate(CCY1)
rate2 = FX.Processor.getRate(CCY2)

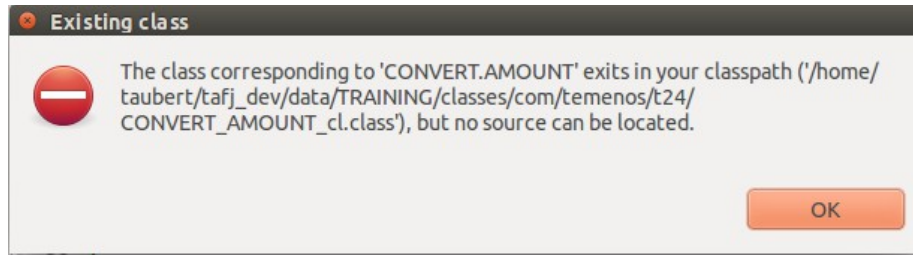
VAL = AMT * rate1 / rate2

RET = ROUND.AMOUNT(VAL, 2)
```

The
last



possibility is that the compiled file is located in your classes / jars, but no source can be found. In that case, and error is shown like this:



Once a method has been implemented using this facility. The code of the method is highlighted. You can un-highlight it with the "R